520.681: Perception for Embodied AI
Homework 2 - Coding
Due: Mar 9, 2026, 11:59pm ET

Instructions

- This is an individual homework, and the coding part is worth 100 points.
- You must submit your solutions via Gradescope.
- Start early! The full training stage takes 1-2 hours to converge . Please plan ahead, and post your questions on Canvas or come to office hours.

Submission

- Submit all Python files you completed as .py files.
- Submit the rendered images for the given test camera transforms in .png format.

All items should be uploaded directly to Gradescope.

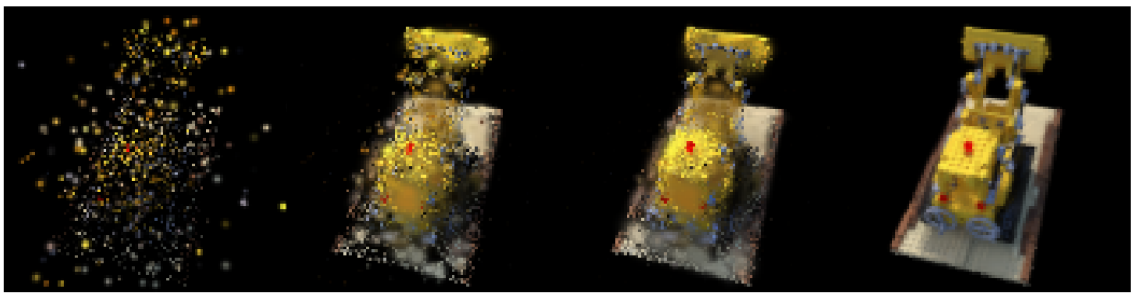

Figure 1: An example of Gaussian Splatting training process.

This assignment provides an introduction to the fundamentals of 3D Gaussian Splatting, which represents a scene explicitly as a dense collection of flexible, anisotropic "blobs." Your goal in this homework is to build a simple Gaussian Splatting pipeline to understand how these 3D geometric primitives are transformed into 2D primitives on the image plane. Please refer to this paper for more details.

1 Gaussian Primitives and Geometric Transforms (35 pts)

First, you will implement the functions for calculating the weights of Gaussians at given pixel coordinates. These include the coordinate transformations that project the Gaussian means to camera space, and the covariance transformations that convert 3D Gaussians to 2D Gaussians in each camera view. In this section, you will complete the code in the file `Gaussian.py`.

1.1 Parameters

3D Gaussians are parameterized by a full 3D covariance matrix Σ and their 3D means μ, defined in world space, as shown in eq. (1):

$$G(x) = \exp(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)) \tag{1}$$

The covariance Σ can be decomposed into a canonical 3D scale factor S and a rotation R, as shown in eq. (2). Optimization is performed on parameters S and R instead of directly on each element of Σ to ensure we maintain a valid positive semi-definite matrix during optimization:

$$\Sigma = RSS^\top R^\top \tag{2}$$

Please finish the function `compute_cov_3d`. Since we optimize quaternions instead of rotation matrices, you can use the helper function `quaternion_to_matrix` from PyTorch3D to convert quaternions to rotation matrices.

1.2 Projection Approximation

In this subsection, we will create functions that perform the "splat"—projecting 3D Gaussians from world space into 2D Gaussians that lie on the image plane of a camera.

The mean μ of each Gaussian is projected to camera space using a standard geometry transform, as shown in eq. (3)

$$^{c}\mu = {}^{c}T_{w}\,{}^{w}\mu \tag{3}$$

$$\mu' = K^{c}\mu \tag{4}$$

You will compute depth from the means in camera space, which will be used later to sort all Gaussians in the rendering function. The Gaussian means in camera space are then projected using camera intrinsics to obtain the 2D coordinates of the Gaussian centers, as illustrated in eq. (4).

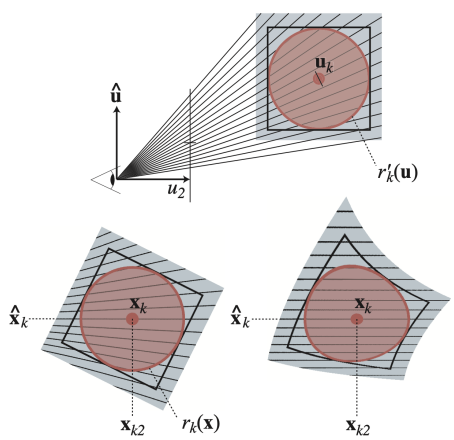

Figure 2: Distortion of 3D Gaussian in 2D image plane. Zwicker et al.

As for covariance, the perspective transformation is non-linear because it involves division by distance (depth). Consequently, the projected Gaussians could be non-ellipsoidal, which increases the complexity of computing weights (see fig. 2). According to Zwicker et al., the projection can be linearly approximated by:

$$\Sigma' = JW\Sigma W^{\top}J^{\top} \tag{5}$$

$$J = \begin{bmatrix} \frac{f_x}{z} & 0 & -\frac{f_x \cdot x}{z^2} \\ 0 & \frac{f_y}{z} & -\frac{f_y \cdot y}{z^2} \end{bmatrix} \tag{6}$$

where (x, y, z) is the Gaussian mean in the camera coordinate system, f_x and f_y are the focal lengths, and W represents the rotation part of the World-to-Camera viewing transformation matrix.

After acquiring the means and variances in each view, you can calculate the Gaussian weights for each pixel by:

$$G'(x) = \exp(-\frac{1}{2}(x - \mu')^\top \Sigma'(x - \mu')) \tag{7}$$

Please complete the functions `compute_means_projection`, `compute_jacobian`, `compute_cov_2D`, and `compute_Gaussian_2D`. Once you finish all the functions in **??**, you can upload the script to Gradescope to get feedback from the autograder.

2 Rendering (35 pts)

2.1 Depth Sorting

Now that we have implemented the basic mathematical properties of the Gaussians, we can build the rasterizer. In this section, you will finish the script `render.py`. First, sort all Gaussians by their depths in ascending order to prepare for alpha blending.

Complete the function `depth_sorting`. You can test it using simple input depth values.

2.2 Alpha Blending

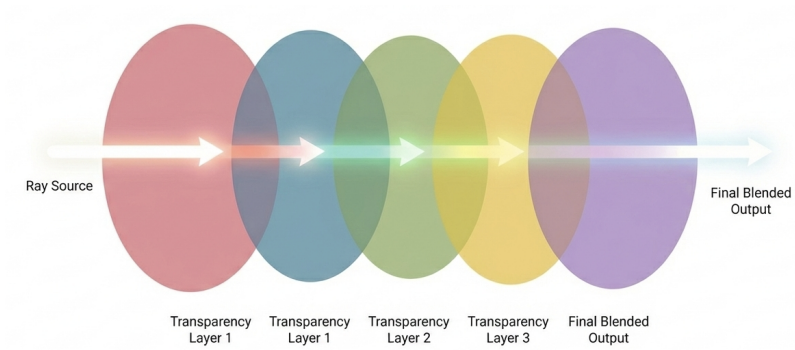

Figure 3: Alpha Blending.

The sorted Gaussians are rendered into images via alpha blending, where the colors of the Gaussians are combined using their alpha weights α_i and the current transmittance T_i:

$$C(x) = \sum_{i=1}^{N} T_i \alpha_i c_i \tag{8}$$

$$\alpha_i = O_i G'(x) \tag{9}$$

$$T_i = \prod_{j=1}^{i-1}(1 - \alpha_i) \tag{10}$$

where $O_i \in [0, 1]$ is the learned per-Gaussian opacity and $c_i \in \mathbb{R}^3$ is the learned per-Gaussian color.

Please complete the function of `compute_alpha` and `compute_transmittance`. And put all functions together to complete the `splat` function.

3 Fitting and Evaluation (30 pts)

3.1 Density Control(10 pts)

We are almost ready to start training. In addition to optimizing current Gaussians, we will adaptively add new Gaussians and prune existing ones to facilitate optimization and reduce sensitivity to initialization. You will finish the density control module, which includes:

- **Densification**: We track the accumulated gradient for each Gaussian. If a small Gaussian has a large gradient, the system creates an exact clone. If a huge Gaussian has a large gradient, the system splits it into two smaller Gaussians. Using the provided splitting and duplicating masks, your task is to copy the affected Gaussian parameters and concatenate them with the original ones.
- **Pruning**: To improve efficiency, the algorithm removes Gaussians that fall below an opacity threshold, as they do not contribute meaningfully to the scene. Given a minimum opacity threshold, your task is to calculate the pruning mask used to remove these invalid Gaussians.

Please complete the functions `_prune` and `_densify`.

3.2 Loss Functions (10 pts)

To train the model, we need a loss function that balances raw pixel accuracy with human-perceived structural similarity. The loss function is a weighted average of the L_1 loss and the SSIM (Structural Similarity Index Measure) loss. You can use built-in PyTorch and TorchMetrics functions to compute these terms:

$$\mathcal{L} = (1 - \lambda)\mathcal{L}_1 + \lambda(1 - \text{SSIM}) \tag{11}$$

where λ is typically set to 0.2.

Please complete the function `calculate_loss` in `gaussian_splatting.ipynb`.

3.3 Evaluation (10 pts)

Finally, you will use the implemented pipeline to fit a simple scene. In this section, you are expected to tune hyper-parameters (e.g., learning rates and training epochs) to generate plausible results. If all functions are implemented correctly, you should see the test loss descend and the rendered image converge toward the ground truth.

PSNR is a commonly adopted metric to measure the similarity of two images. It is defined as:

$$\text{PSNR} = 10 \log_{10} \left(\frac{\max^2}{\text{MSE}} \right) \tag{12}$$

Where max is defined as the maximum value of the images (255.0 or 1.0), and MSE is the mean squared error of 2 images:

$$\text{MSE} = \frac{1}{HW} \sum [I_1(x) - I_2(x)]^2 \tag{13}$$

If your implementations are working correctly, the average PSNR should exceed 30 dB within 4000 training iterations. Once training is complete, the script will save the 10 test views as PNG files. **Please submit these images along with your code to Gradescope**.